

Handout on Working with YACC

1. Communication between scanner and parser

1. The simplest form of communication was achieved in lab 4 experiment, which is summarized below. Note that in this form, “y.tab.h” created by the parser using the %token declarations is used by the scanner. “NUM” that is passed from the scanner is NOT the lexeme but an ASCII value. For single characters, the char literal is directly being passed without using %token to declare it. These are the only forms being communicated from scanner to the parser.

Lex script	Yacc script
<pre>%{ #include "y.tab.h" }% %% {digit}+ {return NUM;} "+" {return '+';} "*" {return '*'};</pre>	<pre>%token NUM %% E: E '+' T {printf(" Reduced E + T to E \n");} T {printf(" Reduce T to E \n");} ; F: NUM {printf(" Reduce NUM to F \n");} ;</pre>

2. Passing the lexeme from the scanner to the parser. This communication needs support in both the scripts.

- The tools agree to use a predefined variable yylval for passing lexemes and other attributes of a token to the parser.
- Usually yylval is declared as “int” in lex.yy.c, but if gcc complains, you have to add this declaration in the lex script as shown below.
- In the lex script, the variable yytext, always stores the lexeme that is recognized, but as a string.
- We want the number that is actually recognized by the scanner, say 2523, to be communicated to the parser. Presently "return NUM" does not achieve it.
- When scanner recognizes the lexeme, say 2523, is is stored internally in yytext as a string "2523". This string can be converted to the number, 2523, by calling the function, atoi() which is defined in C, and converts the string of digits given as argument to its numerical value. Hence atoi(yytext) will produce the number, 2523.
- By writing the C code, "yylval = atoi(yytext);" the numerical value gets saved in yylval which is defined as int, by default.
- How does the parser access this actual lexeme that has been placed in "yylval" by scanner ? The parser accesses the lexeme by using special variables of yacc, which are called \$-variables.
For the rule : $F \rightarrow \text{NUM}$; yacc uses "\$\$" to refer to the LHS nonterminal and "\$1", "\$2", ... to refer to the grammar symbols in the RHS of the rule.
- Given a production rule of the form, the lhs nonterminal is defined or accessed using "\$\$" while each grammar symbol on the rhs that has n symbols, is accessed using variables "\$1" to "\$n", as shown below.

$$F \rightarrow \text{NUM} \{ \text{printf}(\text{" Number is \%d \n"}, \$1); \}$$
$$\$ \$ \quad \$1$$

As a result of this communication between "yylval" and \$-variables, the value of yylval assigned by the scanner will get printed. Yacc adds a default rule of the form, "\$\$ = \$1". Therefore if the action was changed to $F \rightarrow \text{NUM} \{ \text{printf}(\text{" Number is \%d ; \$\$ is \%d \n"}, \$1, \$\$); \}$ will produce an output of the form : "Number is 25 ; \$\$ is 25", provided yylval was set to 25.

- Because of the default action of yacc, in which the LHS \$\$ is always the "\$1" by default (unless explicitly changed by the user),

$$T \rightarrow F$$

$$\$ \$ \quad \$1$$

The value of \$1 is propagated to \$\$, by virtue of which the numeric value associated with F is propagated up as the value of T.

For the second rule, the yylval is available to yacc as \$1 and this value can be used in the C code that is part of the action part of this rule. The action part of the second rule uses "\$1" to access the lexeme corresponding to NUM. If we want this value to be assigned to F, we write "F: NUM {printf(" Reduce %d to F \n", \$1); \$\$ = \$1;}". When F is used in the rhs of a rule, its value can be accessed by "\$i" when F is the ith symbol in the rhs. Understand the reason for the changes and their consequence on the resulting generated scanner and parser.

You should now be able to understand the actions written against the grammar rules in yacc and their purpose.

2. How to construct LALR(1) Parsing Table from the yacc generated file, "y.output"

The contents of the file "y.output" for P1 is reproduced below.

Contents of "y.output"	Interpretation from Theory
Grammar 0 \$accept: E \$end 1 E: E '+' T 2 T 3 T: T '%' F 4 F 5 F: NUM	\$end is used by yacc is the same sense as "\$" used by us in theory \$accept is the augmented start symbol used by yacc, we use E' The rules are copied from the yacc file and numbered from rule 0 to rule 5
Terminals, with rules where they appear \$end (0) 0 '%' (37) 3 '+' (43) 1 error (256) NUM (258) 5	The information is self-explanatory. The token "error" is used by yacc for a special purpose, we do not use this in our course. The numbers in (brackets) are what is communicated from the scanner to the parser.
Nonterminals, with rules where they appear	Similar information as for terminals above, ignored for explanation
DETAILS THAT FOLLOW ARE THE LALR(1) AUTOMATON CONSTRUCTED BY YACC	
State 0 0 \$accept: . E \$end NUM shift, and go to state 1 E go to state 2 T go to state 3 F go to state 4	The contents of state 0 : The item is the kernel LALR(1) item : E' → • E The action entries and the Goto entries follow. The table entry of state 0 is drawn below.

State	Action				Goto		
	%	+	NUM	\$	E	T	F
0	-	-	s1	-	2	3	4

State 1 5 F: NUM . \$default reduce using rule 5 (F)	The contents of state 1 : The kernel item is in bold \$default denotes the remaining terminal symbols and the action is reduce by rule 5 (or r5); no Goto entry
---	--

State	Action				Goto		
	%	+	NUM	\$	E	T	F
0	-	-	s1	-	2	3	4
1	r5	r5	r5	r5	-	-	-

State 3 2 E: T . 3 T: T . '%' F '%' shift, and go to state 7 \$default reduce using rule 2 (E)	The contents of state 1 : The 2 kernel items are in bold on % the action is s7 \$default which denotes the remaining terminal symbols the action is reduce by rule 2
--	--

State	Action				Goto		
	%	+	NUM	\$	E	T	F
0	-	-	s1	-	2	3	4
1	r5	r5	r5	r5	-	-	-
2	-	s6	-	s5	-	-	-
3	s7	r2	r2	r2			

From the description of the other states produced by yacc, the complete parsing table can be constructed. This part is left as an exercise for the audience.